

Programrendszerek fejlesztése

9. gyakorlat

UI/UX megoldások

Összefoglaló tananyag

Szegedi Tudományegyetem – Szoftverfejlesztés Tanszék

2026. tavasz

1. Bevezetés – Web-alkalmazás alapok

Egy modern webalkalmazás három alapvető technológiai rétegből épül fel, amelyeket a „ház analógiával” lehet a legkönnyebben megérteni.

- **HTML (struktúra):** a ház váza, a falak, az alaprajz. Meghatározza, hogy milyen elemek vannak az oldalon (címek, bekezdések, képek, gombok, listák), de semmit nem mond arról, hogyan néznek ki.
- **CSS (megjelenés):** a festék, a bútorok, a dekoráció. Megadja a színeket, méreteket, térközöket, betűtípusokat és az elrendezést. A CSS nélküli HTML oldal funkcionálisan működik, de vizuálisan csúpasz.
- **JavaScript (interaktivitás):** az elektromos hálózat és a kapcsolók. Kezeli a felhasználói eseményeket (kattintás, görgetés, billentyűleütés), dinamikusan módosítja a tartalmat és kommunikál a szerverrel.

Ez a hármas rétegződés a modern frontend keretrendszerekben (Angular, React, Vue) is megjelenik: a komponens template-je a HTML, a stílusfájl a CSS, a komponens osztály/függvény pedig a JavaScript logika.

2. UI és UX fogalmak

A UI és az UX két szorosan összefüggő, de eltérő célú terület.

Felhasználói felület (UI – User Interface)

A UI a vizuális megjelenésre összpontosít: hogyan néz ki az alkalmazás, milyen színeket, betűtípusokat, gombokat és elrendezést használunk. A UI tervezés célja, hogy az alkalmazás esztétikus, tiszta és professzionális megjelenésű legyen.

Felhasználói élmény (UX – User Experience)

A UX a használhatóságra fókuszál: hogyan érzi magát a felhasználó az alkalmazás használata közben, logikus-e a felépítés, könnyű-e megtalálni a funkciókat, és egyértelmű-e, hogy mit kell tenni. A jó UX láthatatlan: a felhasználó nem gondolkodik azon, hogyan kell használni az alkalmazást, mert az intuitívan működik.

A szép felület (UI) önmagában nem elég, ha a felhasználó nem találja meg, amit keres (UX). És fordítva: a logikus felépítés sem segít, ha a felület vizuálisan kaotikus. A kettő együtt adja a jó terméket.

3. UI megközelítések

A különböző képernyőméretekhez való alkalmazkodás három fő megközelítéssel oldható meg.

Adaptív dizájn

Reaktív megközelítés, amely előre definiált képernyőméretekhez (breakpoint-okhoz) igazítja a dizájnt és a tartalmat. Jellemzően 3–5 fix töréspontra terveznek (pl. mobil, tablet, desktop), és minden töréspont egy különálló elrendezést kap. A töréspontok között a megjelenés nem változik.

Reszponzív dizájn

Szintén reaktív megközelítés, de a dizájn és a tartalom folyamatosan, pixel szinten igazodik a képernyő méretéhez. A relatív egységek (% , vw, vh, em, rem) és a rugalmas elrendezési technikák (Flexbox, CSS Grid) biztosítják, hogy az oldal bármilyen szélességen jól nézzen ki, nem csak a fix töréspontokon.

Mobile-first dizájn

Proaktív megközelítés: a tervezés a legkisebb képernyőmérettel (mobil) kezdődik, és onnan bővül a nagyobb képernyők felé. A mobil élmény pontosan ugyanolyan jó, mint az asztali környezet esetén. Ez nem jelent mobil applikációt – a mobile-first egy webes tervezési filozófia, amely a tartalmi prioritásokat a kis képernyő korlátaival keresztül határozza meg.

Fontos: a mobile-first nem egyenlő a mobil applikációval! A mobile-first egy webes szemléletmód, ami a szűkebb képernyő korlátaival kezdi a tervezést.

4. Vizuális hierarchia

A vizuális hierarchia azt határozza meg, hogy a felületen mi tűnik fel először a felhasználónak. A jól megtervezett hierarchia segíti a felhasználót abban, hogy gyorsan azonosítsa a legfontosabb információkat és a végrehajtható akciókat. Négy fő eszközzel alakítható ki:

- **Méret:** a nagyobb elem fontosabbnak tűnik. Egy recept kártya esetén a cím (h3, display font) nagyobb, mint a leírás (body font), ami nagyobb, mint a metaadatok (kis méretű, muted szín).
- **Szín és kontraszt:** az erősebb kontraszt nagyobb figyelmet vonz. A Call To Action (CTA) gomb kiemelése erős háttérszínnel (pl. primary color) arra ösztönzi a felhasználót, hogy arra kattintson.
- **Térköz (whitespace):** a több tér egy elem körül fontosabbá teszi azt. Egy kártya körüli hézag vizuálisan elkülöníti és kiemeli a tartalmat a környezetéből.
- **Tipográfia:** a félkövér szöveg fontosabbnak tűnik, mint a normál, ami fontosabb, mint a light. A betűvastagság skálája (bold > regular > light) közvetlenül kommunikálja az információ fontossági sorrendjét.

Példa: recept kártya felépítése

Egy jól megtervezett recept kártya két fő részből áll. A felső rész tartalmazza a képet (16:9 képarány), rajta egy kiemelő jelvényként (badge) a kategóriát, és opcionálisan egy „saját” jelzést. Az alsó rész az információkat és metaadatokat hordozza: a recept címe címsor

stílusban (h3, display font), alatta a rövid leírás (body font, muted szín, maximum 2 sorra korlátozva), végül a metaadat sor (idő, kalória, nehézség) kis méretű ikonokkal.

5. Design Token rendszer

A design token egy **név-érték pár**, amely a vizuális tulajdonságok legkisebb, névvel ellátott egysége: egy szín, egy méret, egy árnyék, egy betűtípus. A tokenek előre definiált CSS szabályok, amelyek minimalizálják az alkalmazásban előforduló „magic number”-öket (pl. kézzel beírt #1976d2 vagy 16px értékek).

Előnyök

- **Konzisztencia:** rögzített színpaletta az egész alkalmazásban, nincs eltérés komponensek között.
- **Karbantarthatóság:** egy helyen kezelünk mindent – ha a primary szín változik, egyetlen fájlt módosítunk.
- **Témaváltás:** sötét/világos mód egyszerűen cserélhető, mert a tokenek értékeit írjuk felül, nem a komponensek stílusait.
- **Kommunikáció:** a fejlesztő és a designer ugyanazt a nyelvet beszéli – a „primary” mindenki számára ugyanazt a színt jelenti.

Implementáció: CSS Custom Properties

A javasolt megvalósítás CSS Custom Properties (változók) használata, SCSS-ben a :root pszeudo-szelektorban definiálva. Ezáltal a változók a teljes alkalmazásban elérhetők, és bármely komponens stílusfájljából hivatkozhatók.

```
:root {  
  --color-primary: #1976d2;  
  --color-surface: #ffffff;  
  --space-md: 16px;  
  --radius-md: 8px;  
  --shadow-1: 0 1px 3px rgba(0,0,0,0.12);  
}
```

A tokeneket kategóriákra bontjuk: színek, térközök (spacing scale), lekerekítések (radii), árnyékok (shadows), tipográfia (font family, font size, line height, heading méretek) és töréspontok (breakpoints).

6. BEM konvenció

A BEM (Block-Element-Modifier) egy konvencionális nevezéktan a CSS kódban, amely segít a stílusok szervezésében és az ütközések elkerülésében.

- **Block:** a komponens neve. Egy önálló, újrahasználható egység. Példa: recipe-card.

- **Element:** a komponens egy alkotóeleme, amely önállóan nem értelmezhető a block nélkül. Jelölése dupla aláhúzással. Példa: `recipe-card__title`.
- **Modifier:** a block vagy az element egy variációja, amely megváltoztatja a megjelenést vagy viselkedést. Jelölése dupla kötőjellel. Példa: `recipe-card__badge--category`.

A BEM konvenció előnye, hogy a CSS osztálynevek önmagukban dokumentálják a komponens struktúráját, és a lapos (nem mélyen beágyazott) szelektorok révén a specificitás alacsony marad, ami megkönnyíti a felülírásokat és a karbantartást.

```
.recipe-card { } /* Block */
.recipe-card__title { } /* Element */
.recipe-card__badge--category { } /* Modifier */
```

7. Touch target és Line clamp

Touch target (érintési terület)

Az érintési terület az a felület, amelyen belül a felhasználó kattintása vagy érintése aktiválja az interaktív elemet (gombot, linket, ikont). A túl kicsi érintési terület a „kövér ujj” effektust okozza – a felhasználó véletlenül rossz elemre kattint. Ez nem csak kényelmi, hanem akadálymentességi kérdés is.

Platform	Minimum méret	Megjegyzés
Apple (iOS)	44 × 44 pt	Human Interface Guidelines
Google (Android)	48 × 48 dp	Material Design Guidelines

Line clamp (szöveghossz-korlátozás)

A line clamp technika korlátozza, hogy egy szöveges elem hány sorban jelenjen meg. A túlnyúló szöveget három ponttal (ellipsis) vágja le. Ez három UX célt szolgál:

- **Szkennelhetőség:** a felhasználó gyorsan áttekintheti a tartalmat anélkül, hogy elolvasná az összes szöveget.
- **Vizuális ritmus és konzisztencia:** a kártyák azonos magasságúak maradnak, a grid elrendezés nem törik meg.
- **Kognitív terhelés csökkentése:** kevesebb szöveg kevesebb döntést igényel a felhasználotól.

```
.card-desc {
  display: -webkit-box;
  -webkit-line-clamp: 2;
  -webkit-box-orient: vertical;
  overflow: hidden;
}
```

8. Skeleton loading

A skeleton loading (váz betöltés) a hagyományos spinner alternatívája. Ahelyett, hogy egy forgó ikont mutatnánk a betöltés idejére, az oldal elrendezésének „csontvázát” jelenítjük meg szürke helyőrző blokkokkal, amelyek a tartalom végső pozícióját és méretét utánozzák.

Előnyök a spinnerrel szemben

- **Érzékelt sebesség pozitív befolyásolása:** kutatások szerint a skeleton loading mellett a felhasználók gyorsabbnak érzékelik a betöltési időt, mint spinner esetén, mert a vizuális váz azt az érzetet kelti, hogy „már majdnem kész”.
- **Layout Shift elkerülése:** a hirtelen betöltődő tartalom elmozdítja a már látható UI elemeket (Cumulative Layout Shift – CLS). A skeleton lefoglalja a helyet, így a tartalom betöltésekor nem ugrik az oldal.
- **Fókusz irányítása:** a skeleton blokkok mutatják a felhasználónak, hol lesznek az egyes elemek (kép, cím, szöveg), így a szem már a betöltés közben a megfelelő helyre irányul.

9. Angular Material

Az Angular Material a Google által fejlesztett, hivatalos komponenskönyvtár Angular-hoz. A Material Design 3 (M3) specifikációra épül, amely a korábbi verzióknál jóval rugalmasabb tematizálást tesz lehetővé.

Miért használjunk komponenskönyvtárat?

A nulláról felépített UI időigényes és hibalehetőségekkel teli. A komponenskönyvtárak (mint az Angular Material, PrimeNG, MUI, shadcn/ui) előre megírt, tesztelt és akadálymentesített komponenseket adnak. Ez azt jelenti, hogy az ARIA attribútumok, a billentyűzetnavigáció, a reszponzív viselkedés és az egységes vizuális minták (pl. minden dialógus ugyanúgy nyílik és záródik) már készen vannak.

Kulcskomponensek

Az Angular Material komponenseit az alábbi kategóriákba sorolhatjuk a Receptgyűjtemény alkalmazás kontextusában:

Komponens	Kategória	Használat a projektben
MatToolbar	Navigáció	Fejléc (header) komponens
MatCard	Adatmegjelenítés	Receptkártyák a listanézetben
MatChips	Adatmegjelenítés	Kategória szűrők és badge-ek
MatFormField	Űrlapok	Bejelentkezés, regisztráció, értékelés
MatSnackBar	Visszajelzés	CRUD műveletek eredményei
MatDialog	Visszajelzés	Törlés megerősítése

MatButton	Akciók	Minden interaktív gomb
-----------	--------	------------------------

10. Angular Material és Design Tokenek összekapcsolása

Az Angular Material M3 témái a háttérben CSS custom property-ket generálnak – pont úgy, ahogy a saját design token rendszerünk is működik. A két rendszer nem versenyez egymással, hanem rétegződik.

Három rétegű modell

- **1. réteg – Angular Material alapértelmezett tokenek:** a könyvtár saját CSS custom property-jei (pl. `--mat-sys-primary`, `--mat-sys-surface`), amelyeket a telepítéskor automatikusan generál.
- **2. réteg – M3 téma testreszabás:** a `@angular/material` Sass API-ján keresztül (`define-theme` mixin) definiáljuk az egyedi színpalettát, tipográfiát és density-t. Ez felülírja az első réteg alapértelmezett értékeit.
- **3. réteg – Saját design tokenek:** a `:root`-ban definiált CSS custom property-jeink (`--color-primary`, `--space-md`, `--radius-md`, stb.), amelyeket a komponensek SCSS fájljaiban használunk. Ezek kiegészítik a Material tokenjeit a projektspecifikus értékekkel.

A lényeg: az Angular Material adja az alapstruktúrát és az akadálymentesített komponenseket, a saját tokenjeink pedig gondoskodnak arról, hogy az alkalmazás egyedi és konzisztens maradjon. Egy helyen változtatunk, mindenhol frissül.

11. Összefoglalás

A gyakorlat elméleti része az alábbi kulcsfogalmakat és összefüggéseket mutatta be:

- A UI a vizuális megjelenésért, a UX a használhatóságért felel – a kettő együtt ad jó terméket.
- Az adaptív, reszponzív és mobile-first megközelítések a képernyőméretekhez való alkalmazkodás három stratégiája.
- A vizuális hierarchia eszközei (méret, szín, térköz, tipográfia) irányítják a felhasználó figyelmét.
- A design token rendszer a konzisztencia, karbantarthatóság és témaváltás alapja.
- A BEM konvenció szervezett és karbantartható CSS osztályneveket biztosít.
- A touch target és line clamp a mobil eszközökön is jó felhasználói élményt garantál.
- A skeleton loading a spinner helyett az érzékelt sebességet javítja és elkerüli a layout shift-et.
- Az Angular Material előre elkészített, akadálymentesített komponenseket ad, amelyeket a saját design tokenjeinkkel tesztelhetünk.

A gyakorlati részben ezeket az elveket ültettük át a Receptgyűjtemény Angular alkalmazásba az Angular Material telepítésével, a téma konfigurálásával, és a

kulcskomponensek (header, receptkártyák, űrlapok, visszajelzések) Material-alapú megvalósításával.